

1-Klick-Validierung und empirische Kalibrierung der IDEAL/REAL-Graph-prä-OQS-Definition - Pfeiler A (vNext.3-canonical)

antaris, ChatGPT5.2 vs. Copilot

Stand: 22. Dezember 2025

Ziel, Scope und Nicht-Ziele

Ziel. Dieser Plan beschreibt (i) eine *vollautomatische* (“1-Klick”) Python-Validierung der formalen Definition des IDEAL/REAL-Graph im Sinne von *total definiert, deterministisch, implementierbar* (Pfeiler A), sowie (ii) eine *empirische Kalibrierstrategie*, die freie Größen/Optionen iterativ durch Messdaten einschränkt, ohne dabei Kontinuumsraumzeit (Lorentz/Poincaré) als Input vorauszusetzen. Das Ziel ist explizit der Aufbau einer belastbaren *Emergenzbrücke* Richtung Kontinuum (Pfeiler B–H).

Scope.

- **Formale Spezifikation als Quelle der Wahrheit:** REAL-Graph_OQS-Definition.tex (bereitgestellt).
- **Testen:** IDEAL-Trunk (SG/ST p.c.f. Adressstruktur), REAL-Generator (deterministische Umgewichtung), Kron/Schur/DtN-Coarse-Graining, Widerstandsmetrik (inkl. deterministischer $\delta_R I$ -Stabilisierung), sowie die prä-OQS-Schicht (CPTP-Map, repeated interaction).
- **Kalibrieren:** zunächst *elektrotechnisch* über Randantwort/DtN (Multiport-Response), Effektivwiderstände und (optional) RC-Zeitantwort; anschließend (kontrolliert) OQS-Parameter über Zeitreihen.

Nicht-Ziele (zu diesem Zeitpunkt).

- Keine Behauptung globaler Raumzeit-Symmetrien; Symmetrie ist ein *spätes* Approximationsthema (Pfeiler H).
- Keine starke Kontinuum-Identifikation (Manifold) ohne explizites Skalenfenster (Pfeiler F).
- Keine “Erklärung” der Born-Regel; nur operational kompatible Detektor/Response-Modelle als Ziel (Pfeiler G).

1 Prinzipien: Was muss die Validierung respektieren? (Pfeiler A als harte Constraints)

1. **Ontischer Kern:** ontisch bleibt ausschließlich der unendliche IDEAL-Graph; REAL, Coarse-Graining und OQS sind abgeleitet.
2. **Determinismus & Totalität:** jede Auswahlregel (IDs, Ground-Knoten, Regularisierung) ist kanonisch und deterministisch; keine stochastischen Komponenten.

3. **Numerische Regularisierung ist *Numerik*, kein Physik-Parameter:** $\varepsilon_{\text{floor}} = \sqrt{\varepsilon_{\text{mach}}}$, daraus δ_R usw. sind deterministische Float64-Stabilisierung (in der Spezifikation explizit fixiert).
4. **Mikro↔Makro ist operational:** Kron/Schur/DtN muss als überprüfbare, messbare Randantwort implementiert sein. In elektrischen Netzwerken ist die DtN-Map der lineare Operator von Randspannungen zu Randströmen (diskret wie kontinuierlich) [3, 4, 5].

2 1-Klick-Workflow: Reproduzierbarer “Validate+Calibrate”-Runner

2.1 Repository-Layout (minimal, aber CI-tauglich)

```

realgraph/
  requirements.txt
  numerical_policy.yaml
  schemas/
    run_manifest.schema.json
    validation_report.schema.json
  numeric/
    __init__.py
    summation.py          # sum_kahan_lex(values, ids) : mandatory for all reductions
    policy.py             # load+validate numerical_policy.yaml; te_thresholds();
                          policy_hash()==numerical_policy_hash
  linalg/
    __init__.py
    wrappers.py           # det_cholesky(), det_solve_spd(); no iterative solvers
  real/
    __init__.py
    lambda_search.py      # lambda_bracket_bisect(F, policy):
                          deterministic bracket+bisection
  mismatch/
    __init__.py
    mismatch.py           # Mismatch(1/2/3) with fixed conventions
  reporting/
    __init__.py
    report.py             # build validation_report.json;
                          include invalid_events with ids+reasons
  scripts/
    forbid_reductions.py  # pre-commit/Ci:
                          ban sum/np.sum/.sum outside allowlist
    check_no_iterative_solvers.py  # pre-commit/Ci:
                          ban scipy.sparse.linalg iterative solvers
  tests/
    test_summation.py
    test_linalg_wrappers.py
    test_bisection.py
    test_mismatch.py
    test_schema_validation.py
  fixtures/
    run_manifest.example.json
    validation_report.example.json

```

```
.pre-commit-config.yaml
.github/workflows/ci.yml
```

2.2 Deterministische numerische Policy (Runner, nicht-physikalisch)

Verbindliche Summationsimplementierung. Pflichtmaßnahme: Implementiere und exportiere eine zentrale Funktion `sum_kahan_lex(values, ids)` und erzwinge deren Nutzung in *allen* Aggregationen. Dies stellt sicher, dass die deterministische Definition nicht durch assoziative Floating-Point-Effekte (Summationsreihenfolge, BLAS-Reduktionen) unterlaufen wird.

```
def sum_kahan_lex(values, ids):
    # values: iterable[float]; ids: iterable[sortable key]
    pairs = sorted(zip(ids, values), key=lambda t: t[0])
    s = 0.0
    c = 0.0
    for _, x in pairs:
        y = float(x) - c
        t = s + y
        c = (t - s) - y
        s = t
    return s
```

Unit-Tests (Pflicht).

- **Summation determinism:** gleiche $(id, value)$ -Paare in unterschiedlicher Eingangsreihenfolge $\Rightarrow$ `sum_kahan_lex` liefert *bit-identisches* Resultat (innerhalb identischer Toolchain).
- **Summation stability:** Konstruktion von alternierenden Summanden (cancellation) $\Rightarrow$ Kahan-Kompensation reduziert Rundungsfehler gegenüber naiver Summation (als Diagnose, nicht als Gate).

Numerical Policy finalisieren und versionieren.

Pflichtmaßnahme: Ergänze `numerical_policy.yaml` um `Efloor_multiplier`, `AR_multiplier`, `cholesky_tol`, `Csym`, `Crow`, `TE_abs_multiplier` sowie `TE_rel_multiplier` und versioniere die Datei. Der Runner berechnet `numerical_policy_hash := SHA256(canonical_yaml(numerical_policy.yaml))` und schreibt ihn als Pflichtfeld in `run_manifest.json`. Jede Änderung der Policy erzwingt Re-Validation.

Deterministische LA-Wrapper. Pflichtmaßnahme: Implementiere `det_cholesky(A) -> (success, L, reason)` und `det_solve_spd(A,b) -> (x, reason)` mit klarer `Exception→invalid_reason`-Abbildung. Iterative Solver sind per Policy verboten.

```
import numpy as np
import scipy.linalg as sla

def det_cholesky(A):
    # A: expected SPD (after definitional regularization)
    # returns (success, L, reason)
    try:
        L = np.linalg.cholesky(A)    # deterministic call per policy
        return True, L, "ok"
```

```

except Exception as e:
    return False, None, f"chol_fail: {type(e).__name__}"

def det_solve_spd(A, b):
    # returns (x, reason); no iterative solver allowed
    ok, L, reason = det_cholesky(A)
    if not ok:
        return None, reason
    try:
        x = sla.cho_solve((L, True), b, check_finite=False)
        return x, "ok"
    except Exception as e:
        return None, f"solve_fail: {type(e).__name__}"

```

Unit-Tests (Pflicht).

- **Cholesky fail path:** A nicht SPD $\Rightarrow$ `det_cholesky` liefert `success=False` und `reason` beginnt mit `"chol_fail:"`.
- **Solve fail path:** erzwinge Solve-Fehler (z. B. NaNs in b) $\Rightarrow$ `det_solve_spd` liefert `None` und `reason` beginnt mit `"solve_fail:"` (Mapping ist deterministisch).
- **No-iterative guarantee:** statische Prüfung, dass keine iterativen Solver-APIs verwendet werden (Policy/Imports).

Damit “deterministisch/total definiert” in der Praxis nicht durch Implementationsdetails unterlaufen wird, werden alle numerischen Schwellenwerte, Iterationsbudgets und Solverpfade *explicit* in einer `numerical_policy.yaml` festgeschrieben, gehasht (`numerical_policy_hash`) und im Audit-Trail protokolliert. Die Policy ist Teil des Test-Runners (nicht der Physik); **die in der Definition fixierten Regularisierungen** ($\delta_K, \delta_R, \varepsilon_{\text{floor}}, \varepsilon_0$) werden *nicht* “tunable” gemacht, sondern nur *Diagnose- und Akzeptanzbänder* sowie Iterationsbudgets.

Pflicht: globale Summationsstrategie (bindend). Alle skalaren Reduktionen (Energiesummen, Frobenius-Normen, Residuen) laufen strikt in kanonischer ID-Reihenfolge (lexikographisch, Patch P1) mit **kompensierter Summation** (Kahan in float64). Parallelisierte Reduktionen sind verboten. Der Runner erzwingt Single-Thread-Linear-Algebra (z. B. `OMP_NUM_THREADS=1`, `MKL_NUM_THREADS=1`, `OPENBLAS_NUM_THREADS=1`, `NUMEXPR_NUM_THREADS=1`) und loggt die effektiven Werte.

Pflicht: deterministische LA-Wrapper (bindende Fehler-Semantik). Lineare Algebra wird nur über Wrapper aufgerufen; jede Exception wird deterministisch auf `valid=0` bzw. “invalid” gemappt (inkl. `reason`-String). Pivoting/iterative Solver sind *nicht* erlaubt (sonst Stack-/Toleranzabhängigkeit).

- **Float/eps (Definition):** IEEE float64; $\varepsilon_{\text{mach}} := \text{eps}(\text{float64})$, $\varepsilon_{\text{floor}} := \sqrt{\varepsilon_{\text{mach}}}$.
- **Generische Residual-Toleranz (Diagnoseband):** $\tau(A) := C_\tau \varepsilon_{\text{floor}} \max\{1, \|A\|_F\}$ mit Default $C_\tau = 100$.
- **Cholesky-Gate valid (Definitionstreu):** `valid = 1` genau dann, wenn `chol(S +  $\delta_K I$ )` ohne Exception terminiert, mit $S := \frac{1}{2}(M + M^\top)$, $\delta_K := \varepsilon_{\text{floor}} \max\{1, \|S\|_F\}$.
- **Solve-Pfad (Resistance; Definitionstreu):** für $A := L_n^{(g)} + \delta_R I$ wird zuerst Cholesky-Solve versucht; bei Fehlschlag wird `invalid` geloggt und $R_n(u, v) := \infty$ gesetzt (kein “heimlicher” Algorithmuswechsel).

- **Bisection/Bracketing für λ_n (deterministisch):** Parameter $h_0 = 1$, $\gamma = 2$, $N_{\text{expand}} = 30$, $N_{\text{bisect}} = 60$. Stop-Kriterien:

$$|F(\lambda)| \leq \tau_E^{\text{abs}} := \text{TE_abs_multiplier} \varepsilon_{\text{floor}} \max\{1, |E_{\text{target}}|\},$$

$$\frac{|F(\lambda)|}{\max\{1, |E_{\text{target}}|\}} \leq \tau_E^{\text{rel}} := \text{TE_rel_multiplier} \varepsilon_{\text{floor}},$$

mit Defaults $\text{TE_abs_multiplier} = 64$, $\text{TE_rel_multiplier} = 64$. Fehlt ein Sign-Change im Bracket oder wird das Budget ausgeschöpft, gilt deterministischer Fallback: $\lambda_n := 0$ und $\text{invalid_reason} \in \{\text{no_bracket}, \text{budget_hit}\}$.

- **Approx.-Invarianten (Diagnose; *kein* Hard-Gate im Default-Profil):**

$$t_{\text{sym}} := \frac{\|\Lambda - \Lambda^\top\|_F}{\max\{1, \|\Lambda\|_F\}}, \quad t_{\text{row}} := \frac{\|\Lambda \mathbf{1}\|_2}{\max\{1, \|\Lambda\|_F\}},$$

und Akzeptanzbänder $t_{\text{sym}} \leq C_{\text{sym}} \varepsilon_{\text{floor}}$, $t_{\text{row}} \leq C_{\text{row}} \varepsilon_{\text{floor}}$ mit Defaults $C_{\text{sym}} = 10^3$, $C_{\text{row}} = 10^3$.

Beispiel: numerical_policy.yaml (Pflichtdefaults).

```
numerical_policy_version: 2
float: float64

# --- Threading: deterministisch (kein parallel reduction) ---
threads:
  OMP_NUM_THREADS: 1
  MKL_NUM_THREADS: 1
  OPENBLAS_NUM_THREADS: 1
  NUMEXPR_NUM_THREADS: 1

# --- Bindende Summationsregel (alle Aggregationen) ---
summation:
  strategy: sum_kahan_lex    # bindend: sum_kahan_lex(values, ids)

# --- Floors/Multiplikatoren (nur für Normalisierungen/Checks;
# nicht für definitorische delta-Formeln) ---
floors:
  Efloor_multiplier: 1.0    # Efloor := Efloor_multiplier * eps_floor(float64)

# --- Widerstandsmetrik/Diagnostik ---
resistance:
  AR_multiplier: 1.0        # Diagnose-Multiplikator (keine Modelländerung)

# --- DtN/Response-Residuen (Diagnose; Default: kein CI-Fail) ---
dtn_residuals:
  Csym: 1.0e3                # Bandfaktor für t_sym
  Crow: 1.0e3                # Bandfaktor für t_row
  gate_default: false        # hard-gate nur in strict_numeric-Profil

# --- lambda-Bracketing/Bisection (deterministisch) ---
bisection:
  h0: 1.0
```

```

gamma: 2.0
Nexpand: 30
Nbisect: 60
TE_abs_multiplier: 64
# tau_E^abs := TE_abs_multiplier * eps_floor * max(1, |E_target|)
TE_rel_multiplier: 64      # tau_E^rel := TE_rel_multiplier * eps_floor

# --- Lineare Algebra: deterministische Wrapper; iterative Solver verboten ---
linear_algebra:
  cholesky_tol: 1.0e2      # Residual-/Warn-Schwelle (Diagnose)
  allow_iterative: false
  wrappers:
    det_cholesky: true
    det_solve_spd: true

```

2.3 “1-Klick”-Entry-Point

- `make validate`: komplette Spec-Suite (inkl. Golden Master) auf definiertem Parameter-Grid.
- `make calibrate`: lädt Messdatensätze, führt Fits durch, erzeugt Report (LaTeX/Markdown), aktualisiert Parameter-Priors.
- `make all`: `validate` → `calibrate` → `re-validate` (mit kalibrierten Parametern), mit strikten Pass/Fail-Kriterien.

Design-Entscheidung (best choice für Pfeiler B–H). Die zentrale kalibrierbare Schnittstelle ist die *Randantwort* (DtN/Response-Matrix): Sie ist (a) in klassischer Elektrophysik direkt messbar, (b) mathematisch eng an Schur/Kron gekoppelt [8, 3], und (c) unabhängig von Kontinuumsraumzeit-Symmetrien.

3 Validierungs-Suite: Was wird getestet?

3.1 CI-Gate-Matrix: Hard-Gates vs. Diagnose je Runner-Profil

Damit die “1-Klick”-Suite in der Praxis reproduzierbar bleibt, muss der Runner *explizit* festlegen, welche Checks als **Hard-Gate** (CI-Fail) gelten und welche nur **Diagnose** (WARN im Report) sind. Wichtig: Residuen zu Λ -Invarianten (Symmetrie-/Zeilensummen) dürfen wegen der definitorischen Regularisierung (z. B. δ_K) *nicht* als harte Algebra-Identitäten behandelt werden, sondern werden standardmäßig als Diagnose geführt.

Legende:

- **FAIL** = CI bricht ab (Hard-Gate).
- **WARN** = im Report markieren, aber CI läuft weiter (Diagnose).
- **SKIP** = in diesem Profil nicht ausgeführt.
- **PASS*** = Test besteht, aber setzt `invalid_flags/invalid_events` (z. B. definierter Fall-back).

Check (Kurz)	Core	Strict	OQS small	OQS full	Longrun	Sweep
T0 Spec-Lock (Hashes, Versions, Stack-Fingerprint vorhanden)	FAIL	FAIL	FAIL	FAIL	FAIL	FAIL
Schemas valide (Mani-fest/Report via <code>jsonschema</code>)	FAIL	FAIL	FAIL	FAIL	FAIL	FAIL
Numerical-Policy geladen, <code>numerical_policy_hash</code> geloggt	FAIL	FAIL	FAIL	FAIL	FAIL	FAIL
Hook/CI-Scan: keine direkten Reduktionen (<code>sum/np.sum</code>)	FAIL	FAIL	FAIL	FAIL	FAIL	FAIL
Hook/CI-Scan: keine iterativen Solver (Policy)	FAIL	FAIL	FAIL	FAIL	FAIL	FAIL
Golden-Master (kleine toolchain-equal)	FAIL	FAIL	FAIL	FAIL	SKIP	SKIP
DtN global/local berechnen, <code>valid-Gates/Fallbacks</code> korrekt	FAIL	FAIL	FAIL	FAIL	SKIP	SKIP
Λ -Residuen $t_{\text{sym}}, t_{\text{row}}$ (nur Diagnose)	WARN	FAIL	WARN	WARN	WARN	WARN
Widerstandsmetrik: Solve-Pfad (SPD) + Disconnection-Regeln	FAIL	FAIL	FAIL	FAIL	SKIP	SKIP
Option-Quercheck (Option 1 vs. 2)	WARN	FAIL	WARN	WARN	WARN	WARN
λ -Search: Root gefunden	PASS*	FAIL	PASS*	PASS*	PASS*	PASS*
Fallback-Rate > 0 (definiert, aber Qualitätswarnung)	WARN	FAIL	WARN	WARN	WARN	WARN
OQS: Choi-PSD (Choi-Eigenwerte)	SKIP	SKIP	FAIL	SKIP	SKIP	SKIP
OQS: TP + Positivität (Sampling / PSD-Flooring-Band)	SKIP	SKIP	FAIL	FAIL	SKIP	SKIP
Monitoring-Heuristiken (Drift, Metastabilität)	SKIP	SKIP	SKIP	SKIP	WARN	SKIP
Parameter-Sweeps (Grid+Seeds, deterministisch)	SKIP	SKIP	SKIP	SKIP	SKIP	WARN

Profil-Definitionen (minimal, auditierbar).

- **Core:** T0–T10 ohne OQS-Choi; Λ -Residuen als Diagnose.
- **Strict:** wie Core, aber Diagnosebänder werden als Gate behandelt; Fallbacks sind CI-Fail.
- **OQS small:** zusätzlich Choi-PSD/TP-Checks nur für sehr kleine Dimensionen (z. B. $L_{\text{max}} \leq 2$).
- **OQS full:** keine Choi-PSD; TP/Positivität über Sampling.
- **Longrun:** mehrstufige Läufe, Monitoring als Diagnose.
- **Sweep:** deterministische Grid-Sweeps, Ergebnisse als Diagnose (CI optional separat).

Hinweis zur Determinismus-Aussage. Bit-identische Ergebnisse sind als Hard-Gate *nur* für dieselbe Toolchain/BLAS-Konfiguration sinnvoll. Deshalb muss der Runner den Stack (Python/Numpy/SciPy, Threads) im Manifest vollständig loggen; bei Abweichung ist der entsprechende Golden-Master-Check zu **SKIP** degradieren (nicht “FAIL”).

3.2 T0: Spec-Lock und Audit-Trail

1. **Spec-Fingerprint:** SHA256 über `REAL-Graph_OQS-Definition.tex`. Jede CI-Run referenziert diesen Hash.
2. **Konfig-Fingerprint:** YAML/JSON-Config wird normalisiert (sort keys) und ebenfalls gehasht.
3. **Audit-Trail:** jeder Test schreibt (i) verwendete Parameter, (ii) deterministische Selektionsentscheidungen, (iii) numerische Konditionszahlen/Restfehler.
4. **Schemas (Manifest/Report; deterministisch):** Der Runner schreibt `run_manifest.json` und `validation_report.json` mit kanonischer Key-Sortierung. Mindestfelder (Pflicht, typisiert): `{spec_sha256, plan_sha256, config_hash, numeric_policy_hash, stack_fingerprint, profiles_run, golden_masters, invalid_flags, residuals, artifacts}`.

CI-Pflicht: Schema-Validation.

```
python -m jsonschema -i run_manifest.json schemas/run_manifest.schema.json
python -m jsonschema -i validation_report.json schemas/validation_report.schema.json
```

Ein fehlendes/inkonsistentes Schema ist ein *CI-Fail*, da es den Audit-Trail bricht. [1]

3.3 T1: IDEAL-Trunk (SG/ST) — Strukturtests

- **Adressraum & Quotient (constant-tail):** eindeutige Repräsentanten, kanonische lex. Ordnung, vollständige Abdeckung.
- **Approximanten G_L :** korrekte Knoten-/Kantenanzahl, Rand B_0 korrekt, Einbettung nur zu Visualisierung.
- **Ordnungs-Fixture (Patch P1; CI-Gate):** Alle Randvektoren/-matrizen verwenden strikt die Patch-1-Randordnung $(b_i)_{i \in S_\bullet}$. Alle Subsets/Subset-Listen (z. B. bei principal submatrices und Schur-Blöcken) werden strikt in aufsteigender id-Ordnung verarbeitet (*Subset-Ordnung als Fixture*). Das wird explizit getestet, da sonst Golden-Master/Hashes fragil werden. [1]
- **Energie-Renormierung (SG):** Validierung gegen bekannte Dirichlet-Form-Renormierung am SG (z. B. Faktor $r = 3/5$ in Standard-Harmonic-Structure) [13, 12].

3.4 T2: REAL-Update (vNext.3-canonical) — Determinismus, Numerik-Scope, Pflichtpfade

Dieser Block ist *Definitionstreue*: alle Regularisierungen und Fallback-Pfade (invalid/ λ -Fallback) werden als *Teil der Definition* getestet, nicht als Implementationsdetail.

T2A Determinismus (praktisch): Gleiche Inputs liefern gleiche Outputs *für denselben eingefrorenen Stack* (Python/numpy/scipy/BLAS/LAPACK) in IEEE float64, bei festgelegter Iterations-/Summationsordnung (lexikographische ID-Ordnung; keine parallelisierten Reduktionen). Cross-Platform wird *nicht* als bit-identisch behauptet; stattdessen werden Hashes plus Residual-/Toleranzbänder als Akzeptanzkriterium verwendet. Der Run schreibt einen `stack_fingerprint` (Versionen, BLAS, OS/CPU) in den Audit-Trail und setzt für CI deterministisch `OMP_NUM_THREADS=1` (analog MKL/OpenBLAS Threads), um nicht-deterministische Reduktionsreihenfolgen zu vermeiden. [1, 2]

T2B Leitwert-Update (alle Kanten) + Randenergie-Erhaltung (kein Freeze): Pflichttest der exakt spezifizierten Zweistufigkeit: (i) provisorischer Schritt $\tilde{c}_{n+1}(e) = c_n(e) \exp(\Delta_n(e))$ für alle $e \in E$; (ii) Rand-Normalisierung über λ_n gemäß Definition:

$$c_{n+1}^{(\lambda)}(e) = \begin{cases} \tilde{c}_{n+1}(e) \exp(\lambda), & e \in E_\partial, \\ \tilde{c}_{n+1}(e), & e \notin E_\partial, \end{cases} \quad c_{n+1}(e) := c_{n+1}^{(\lambda_n)}(e).$$

Es gibt *keinen* Sonderfall, der Kanten (z. B. E_0) “festfriert”. [1]

T2C Numerik-Scope als Definition (float64): $\varepsilon_{\text{mach}} := \text{eps}(\text{float64})$, $\varepsilon_{\text{floor}} := \sqrt{\varepsilon_{\text{mach}}}$. Pflichttests:

(i) $\delta_{K,\text{glob}}$ und $\delta_K(n, w)$ werden exakt als $\varepsilon_{\text{floor}} \max\{1, \|S\|_F\}$ berechnet, wobei S der *symmetrisierte* Innenblock ist (SII-Policy) und die Solve-/Cholesky-Pfade deterministisch geloggt werden; (ii) δ_R wird exakt als $\varepsilon_{\text{floor}} \max\{1, \|L^{(g)}\|_F\}$ berechnet (kein physikalischer Parameter). Abweichungen sind ein **Hard-Fail** (Spec-Drift). [1, 2]

T2D Deterministische Logging-Semantik: Jeder “invalid”-Fall (z. B. $\text{valid}(n, w) = 0$, λ -Fallback) muss (i) im JSON-Report erscheinen, (ii) den deterministischen Grund kodieren (`chol_fail`, `no_bracket`, `budget_hit`, ...), und (iii) die betroffenen Zellen/Kanten in kanonischer Ordnung listen. [1]

3.5 T3: Driver-B & Coarse-Graining via Kron/Schur/DtN — global + lokal, Mismatch, λ_n -Constraint

Die Kron-Reduktion ist die Schur-Komplement-Reduktion der (loopy) Laplace-Matrix auf eine Randmenge α [8]. In diskreten Resistor-Netzwerken entspricht die Randantwort (Response-Matrix) der diskreten Dirichlet-to-Neumann-Map [3]. Wichtig: durch die expliziten δ_K -Regularisierungen sind einige “Invarianten” nur *näherungsweise* zu erwarten; Residuen werden mit Toleranzbändern protokolliert (kein falscher Hard-Gate). [1, 2]

T3A Globales DtN Λ_n auf B_0 : Implementierte DtN = Schur-Komplement (gemäß (R4) in der Definition), *inkl.* SII-Symmetrisierung und $\delta_{K,\text{glob}}$. Zusätzlich werden *normierte Residuen* als reine Diagnostik geloggt:

$$t_{\text{sym}} := \frac{\|\Lambda_n - \Lambda_n^\top\|_F}{\max\{1, \|\Lambda_n\|_F\}}, \quad t_{\text{row}} := \frac{\|\Lambda_n \mathbf{1}\|_2}{\max\{1, \|\Lambda_n\|_F\}}.$$

Wichtig: wegen der definitorischen Regularisierung (δ_K) sind Symmetrie/Row-Sum nur $\approx$ -Invarianten. **Default-Regel im Runner/CI:** diese Residuen sind *Diagnose* und dürfen *nicht* als CI-Fail konfiguriert werden (`gate_dtn_residuals=false`). Optionales Profil `strict_numeric` darf bandbasierte Gates aktivieren, z. B. $t_{\text{sym}} \leq C_{\text{sym}} \varepsilon_{\text{floor}}$, $t_{\text{row}} \leq C_{\text{row}} \varepsilon_{\text{floor}}$ mit explizit gehashter Policy (keine stillen Defaults). [1, 2, 3]

T3B0 Corner-Konventionen $w = \emptyset$ (Patch P2; Regression): Pflichttest der Totalitäts-/Corner-Fixierungen:

$$V_\emptyset = B_0, \quad I_\emptyset = \emptyset, \quad \text{valid}(n, \emptyset) = 1, \quad \Lambda_{n, \emptyset} = (L_n)_{B_0, B_0}, \quad E_\emptyset = \{e \in E : \ell(e) = 0\}.$$

Außerdem Corner-Override: $\kappa_n^q(\emptyset) := \kappa_n^{\text{geom}}(\emptyset)$ (wie spezifiziert, da $m_n(w)$ für $w = \emptyset$ nicht definiert ist). Dieser Test ist audit-klar als eigener Regression-Fall zu führen. [1]

T3B Lokales DtN $\Lambda_{n,w}$ + Gate $\text{valid}(n, w)$ (Pflichtpfad): Baue $Q_{n,w}$ als principal submatrix gemäß Definition (Shunts), setze I_w und prüfe:

- $I_w = \emptyset \Rightarrow \Lambda_{n,w} := (Q_{n,w})_{BB}$ (deterministischer Corner-Fall).
- sonst: $\text{valid}(n, w) = 1$ genau dann, wenn Cholesky auf dem regulierten, symmetrisierten Innenblock gelingt; nur dann Schur-Komplement, anderenfalls Fallback $\Lambda_{n,w} := (Q_{n,w})_{BB}$.

Die Suite muss beide Pfade (valid/invalid) mit konstruierten Testfällen triggern. [1]

T3C Mismatch(1/2/3), anchored trace-Normierung, m_B -Default: Pflichttests (Definitionstreu; *keine* impliziten Normkonventionen):

- (a) **Anchored trace:** für jede Zelle w mit Randordnung $B_w = (b_{w,i})_{i \in \mathcal{S}_\bullet}$ (Patch P1) und $m_B \in \{1, \dots, |B_w|\}$ definiere

$$\text{tr}_{\text{anch}}(\Lambda_{n,w}) := \sum_{i=1}^{m_B} (\Lambda_{n,w})_{ii}, \quad \hat{\Lambda}_{n,w} := \frac{\Lambda_{n,w}}{\max\{\text{tr}_{\text{anch}}(\Lambda_{n,w}), \varepsilon_{\text{floor}}\}}.$$

Default: $m_B = 1$. Der Plan verlangt explizite Tests für $m_B = 1$ und mindestens einen weiteren zulässigen Wert.

- (b) **Mismatch(1):** $\text{Mismatch}_n^{(1)}(w) := \|\hat{\Lambda}_{n,w} - \hat{\Lambda}_{0,w}\|_F$.
- (c) **Mismatch(2):** bilde die Distanzmatrix $D_{n,w}$ aus den Widerstands-Distanzen auf B_w , $(D_{n,w})_{ij} := d_{n,w}(b_{w,i}, b_{w,j})$, normiere $\hat{D}_{n,w} := D_{n,w} / \max\{\|D_{n,w}\|_F, \varepsilon_{\text{floor}}\}$ und setze $\text{Mismatch}_n^{(2)}(w) := \|\hat{D}_{n,w} - \hat{D}_{0,w}\|_F$.
- (d) **Mismatch(3):** symmetrisiere $\hat{\Lambda}_{n,w}^{\text{sym}} := \frac{1}{2}(\hat{\Lambda}_{n,w} + \hat{\Lambda}_{n,w}^\top)$, berechne die Eigenwerte $e(\hat{\Lambda}_{n,w})$ (aufsteigend sortiert, feste Konvention), analog $e(\hat{\Lambda}_{0,w})$, und setze $\text{Mismatch}_n^{(3)}(w) := \|e(\hat{\Lambda}_{n,w}) - e(\hat{\Lambda}_{0,w})\|_2$.
- (e) **Domänenregeln:** Wenn ein erforderlicher Input undefiniert ist (z.B. Disconnection $\Rightarrow \infty$ -Distanzen), wird der Mismatch-Wert als N/A markiert (mit **reason**); es gibt keine stillen Defaults.

[1]

T3D κ -Sättigung inkl. valid-Override + Corner-Override: Pflichttests: κ^{geom} wird bei $\text{valid}(n, w) = 0$ auf 0 überschrieben; Corner-Override $\kappa^q(\emptyset) := \kappa^{\text{geom}}(\emptyset)$ wird exakt eingehalten.

[1]

T3E $\tilde{p}$ -Zentrierung (Totalität für leere Mengen): Pflichttests: $\text{pen}_{n,w} \equiv 0$ falls $|E_w^{(k)}| = 0$ und analog für $\emptyset$; keine Division-by-zero-Pfade. Leere-Mengen-Fälle sind als CI-Regression explizit enthalten. [1]

T3F Update-Exponent $\Delta_n(e)$ inkl. $K_{\text{max}} = 0$ und Corner-Fallbacks: Pflichttest: Zwei-Terme-Struktur (Skalensumme + Corner-Term) wird exakt implementiert; bei $K_{\text{max}} = 0$ ist die Summe leer und nur der Corner-Term kann wirken (sofern $E_\emptyset \neq \emptyset$). Corner/Emptyset-Kantenfälle sind Regressionstests. [1]

T3G λ_n -Randenergieerhaltung (Bracketing+Bisektion + deterministischer Fallback): Pflichttests: (i) definierte Energiegleichheit wird via festem Budget (Bracketing/Bisektion) angestrebt; (ii) bei Nicht-Konvergenz: $\lambda_n := 0$ und invalid-Markierung; (iii) **Bracket-Diagnostik:** im Report muss stehen, ob ein Sign-Change im Bracket gefunden wurde (No-Bracket wird nicht “wegerklärt”, sondern explizit geloggt). [1, 2]

Deterministischer Bracketing-/Bisektions-Pfad (Pseudo-Code, CI-testbar). Wir fixieren $F(\lambda) := E_{\text{tot}}^\partial(n+1; \lambda) - E_{\text{target}}$. Mit $\varepsilon_{\text{floor}} := \sqrt{\varepsilon_{\text{mach}}(\text{float64})}$ und den Policy-Konstanten TE_abs_multiplier , TE_rel_multiplier setzen wir

$$\tau_E^{\text{abs}} := \text{TE_abs_multiplier} \varepsilon_{\text{floor}} \max\{1, |E_{\text{target}}|\}, \quad \tau_E^{\text{rel}} := \text{TE_rel_multiplier} \varepsilon_{\text{floor}}.$$

Der deterministische Algorithmus lautet (symmetrisches Expand um 0, dann Bisektion):

```

def lambda_bracket_bisect(F, h0, gamma, Nexpand, Nbisect, tau_abs, tau_rel):
    a = 0.0
    Fa = F(a)
    if abs(Fa) <= max(tau_abs, tau_rel):
        return a, "ok_at_zero"

    h = h0
    bracket = None
    for j in range(Nexpand):
        b1, b2 = +h, -h
        Fb1, Fb2 = F(b1), F(b2)
        if Fa*Fb1 < 0.0:
            bracket = (a, b1, Fa, Fb1); break
        if Fa*Fb2 < 0.0:
            bracket = (b2, a, Fb2, Fa); break
        h *= gamma

    if bracket is None:
        return 0.0, "no_bracket"

    lo, hi, Flo, Fhi = bracket
    for i in range(Nbisect):
        mid = 0.5*(lo+hi)
        Fm = F(mid)
        if abs(Fm) <= max(tau_abs, tau_rel):
            return mid, "ok"
        if Flo*Fm < 0.0:
            hi, Fhi = mid, Fm
        else:
            lo, Flo = mid, Fm
    return 0.0, "budget_hit"

```

Unit-Tests (Pflicht).

- **Bisection ok:** Konstruiere eine monotone Testfunktion F mit garantiertem Sign-Change in einem bekannten Intervall. Akzeptanz: `reason=="ok"` und $|F(\hat{\lambda})| \leq \max(\tau_E^{\text{abs}}, \tau_E^{\text{rel}})$.
- **Bisection ok_at_zero:** Wähle $F(0) = 0$ (bis auf floating-point noise). Akzeptanz: `reason=="ok_at_zero"` und $\hat{\lambda} = 0$.
- **Bisection no_bracket:** Wähle F ohne Sign-Change (z.B. konstantes positives F). Akzeptanz: `reason=="no_bracket"`, deterministischer Fallback $\lambda_n := 0$, `invalid_reason="no_bracket"` wird geloggt.
- **Bisection budget_hit:** Wähle F mit extrem flachem Verlauf oder setze N_{bisect} absichtlich klein, so dass die Stoppbedingung nicht erreicht wird. Akzeptanz: `reason=="budget_hit"`, deterministischer Fallback $\lambda_n := 0$, `invalid_reason="budget_hit"` wird geloggt.
- **Determinismus:** Wiederhole alle Fälle mit gleicher Policy/Seed/Stack-Fingerprint. Akzeptanz: identische Reasons, identische $\hat{\lambda}$ (bitwise innerhalb identischer Toolchain) und identische Logs/Hashes.

3.6 T4: Widerstandsmetrik — grounded solve (Option 2, 1:1) + Diagnostik

Effektiver Widerstand ist ein zentraler Graph-Observable (Resistance distance) [11, 9, 10]. Die Definition bevorzugt Option 2 (grounded solve) mit deterministischem Jitter δ_R [1].

- **Grounded Solve-Definition (1:1):** deterministische Wahl des Ground-Knotens $g := \arg \min_{x \in V} \text{id}(x)$, reduzierte Laplacian $L_n^{(g)}$, $\delta_R := \varepsilon_{\text{floor}} \max\{1, \|L_n^{(g)}\|_F\}$, Solve $(L_n^{(g)} + \delta_R I)x = b$, $R_n = b^\top x$, $d_n = \sqrt{R_n}$. Keine explizite Symmetrisierung im Solve-Pfad (Diagnostik separat). [1]
- **Solverpfad (bindend):** für $A := L_n^{(g)} + \delta_R I$ wird ausschließlich ein direkter SPD-Solve verwendet (Cholesky-Faktorisierung). Fehlschlag der Faktorisierung oder Solve-Residual oberhalb $\tau(A)$ (vgl. Sec. 2.2) setzt deterministisch **invalid** und $R_n := \infty$ (keine iterative “Rettung”). Der gewählte Pfad und Residuen werden geloggt.
- **Diagnostik (ohne Einfluss auf Definition):** logge $r_{\text{asym}} := \|L_n^{(g)} - (L_n^{(g)})^\top\|_F$ sowie Solve-Residuen $\|(L_n^{(g)} + \delta_R I)x - b\|_2$.
- **Quercheck (optional; Small-Profile, rein diagnostisch):** Zusätzlich zum definitiven Grounded-Solve wird für kleine Instanzen ein Quercheck gerechnet, ohne die Definition zu ändern:
 - *Ground-Varianten-Check:* wähle einen zweiten Ground g' (z. B. zweitkleinste ID) und berechne $R_n^{(g)}(u, v)$, $R_n^{(g')}(u, v)$.
 - *Option-1-Check (falls implementiert):* berechne $R_n^{\text{pinv}}(u, v)$ via Pseudoinverse/Linearsolve auf dem Vollsistem (nur Small-Profile).

Akzeptanz erfolgt mit *konditions- und regularisierungsgekoppelter* Toleranz (kein fixer Grenzwert): Setze $A := L_n^{(g)} + \delta_R I$ und eine Konditionsschätzung $\kappa(A)$ (z. B. cond_2 im Small-Profile). Dann

$$|R_{\text{ref}} - R_{\text{chk}}| \leq \tau_R := C_R \varepsilon_{\text{floor}} \kappa(A) \max\{1, |R_{\text{ref}}|\}, \quad C_R \text{ Default} = 10^3,$$

und logge $\kappa(A)$, δ_R , τ_R als Diagnosegrößen. [2]

- **Disconnection:** falls der Solve scheitert bzw. der Graph nicht verbunden ist: $R_n = \infty$ (und damit $d_n = \infty$) wird deterministisch kodiert und im Export als Sonderwert markiert (kein stilles Clipping). [1]

3.7 T5: Prä-OQS-Schicht — Φ_{B_0} , repeated interaction, Instrument-Default, quantum-first Readout

Die Definition enthält explizit eine prä-OQS-Schicht: (i) der CPTP-Randkanal Φ_{B_0} , (ii) der repeated-interaction Zeitschritt $\mathcal{T}_n$ via Stinespring-Dilation, (iii) $\Psi_n = \Phi_{B_0} \circ \mathcal{T}_n$, sowie (optional) ein deterministisches Instrument $\mathcal{M}_n$ mit Default $\mathcal{Y}_n = \{0\}$, $K_{n,0} = I$ [1]. Damit diese Inhalte nicht nur “erwähnt”, sondern 1:1 testbar abgedeckt sind, definiert die Suite ein Minimalprofil **oqs_min**: $d_E = 1$, $\sigma_E = I$, Δt fix, $H_E = 0$, $H_{\text{int},n} = 0$ (kanonische Defaults). Damit der 1-Klick-Runner nicht an einer Choi-Diagonalisierung skaliert, gibt es zwei Validierungsprofile:

- **oqs_small:** kleine Dimensionen (z. B. $L_{\text{max}} \leq 2$, $d_p \leq 2$, optional $d_E \leq 2$): **volle** Choi/PSD-Checks für Φ_{B_0} (und optional Ψ_n).
- **oqs_full:** produktive Größen: TP-Checks und *strukturelle* CP-Checks (Kraus-/Stinespring-Konstruktion) plus Positivität über Stichprobenzustände.

Hard-Gate vs. Skalierung (Choi). Choi-PSD ist ein Pflichttest *ausschließlich* im Profil `oqs_small`. Im Profil `oqs_full` ist `choi_psd=false` verpflichtend; andernfalls ist die 1-Klick-Suite praktisch nicht skalierbar, ohne dass die Definition dies verlangt. [1]

1. **Φ_{B_0} exakt wie definiert:** Implementiere $\Phi_{B_0}(\rho) = P\rho P + \text{Tr}((I - P)\rho)\sigma_{B_0}$ mit $\sigma_{B_0} = P/\text{Tr}(P)$. Pflichttests: (a) Trace-Preservation (numerisch), (b) CP-Check skaliert nach Profil: `oqs_small` via Choi-PSD; `oqs_full` via expliziter Kraus-Konstruktion (aus der Spektraldekomp. von σ_{B_0}) und Vollständigkeit $\sum_j K_j^\dagger K_j = I$ plus Positivitäts-Stichproben, (c) Idempotenz $\Phi_{B_0} \circ \Phi_{B_0} = \Phi_{B_0}$ (bis Toleranz). [15, 16, 17, 18, 1]
2. **Unitary U_n und Kanal $\mathcal{T}_n$:** Im Minimalprofil ist $U_n = \exp(-i\Delta t(H_n \otimes I))$ mit $H_n = L_n \otimes I_{d_p}$ (und Defaults), also unitär. Pflichttests: $\|U_n^\dagger U_n - I\|_F \leq \tau_U$ (smoke) und Trace-Preservation/Positivität von $\mathcal{T}_n(\rho) = \text{Tr}_E(U_n(\rho \otimes \sigma_E)U_n^\dagger)$ für Stichprobenzustände. [1, 2]
3. **Komposition $\Psi_n = \Phi_{B_0} \circ \mathcal{T}_n$:** Pflichttests: (a) $\text{Tr}(\Psi_n(\rho)) \approx 1$, (b) Eigenwerte $\geq -\tau$, (c) Kontraktionssanity (Trace-Distance nimmt nicht zu) als numerischer Check; `oqs_small`: zusätzlich Choi-PSD für Ψ_n . [19, 20]
4. **Instrument-Default & Readout $m_n(w)$:** Pflichttest: Default $\mathcal{Y}_n = \{0\}$, $K_{n,0} = I \Rightarrow \bar{\rho}_n = \rho_n$ wird exakt umgesetzt. Readout: $m_n(w) = \text{Tr}(\Pi_w \bar{\rho}_n)$ liegt in $[0, 1]$ und ist für deterministische Testzustände reproduzierbar; $\kappa_n^q(w) = \tanh(m_n(w))$ wird als (0,1)-beschränktes Signal erzeugt. [1]

3.8 T6: Golden Master / Regression

- Fixiere kleine Levels ($L_{\max} \in \{0, 1, 2, 3\}$) und Exportformate (Graph+Leitwerte+DtN+R-Metrik+OQS-Step).
- Schreibe deterministische Referenzartefakte (NPZ/JSON) und vergleiche hashbasiert.
- Jede Änderung am Kern erfordert Update-Note + erneute vollständige Validierung.

3.9 T7–T12: Monitoring, Sensitivität, Sweeps (erweiterte Profile)

Die Tests T0–T6 sind die *harten* Spec- und Infrastrukturtests (Definitionstreue + deterministische Numerik). Die erweiterte Suite T7–T12 umfasst deterministische Laufzeitprofile, Monitoring und Sensitivitäts-/Sweep-Jobs, die in CI als getrennte Stufen laufen (nicht jeder 1-Klick-Run muss alle Sweeps ausführen):

- **T7** Langlauf-Profil und Drift/Fixed-Point-Heuristiken (Fallback-Raten, Konvergenzindikatoren).
- **T8** Sensitivitätsanalyse (deterministische Grids/Seeds; Report der Stabilitätsfenster).
- **T9** CI-Hard-Gates: Schema-Validation, Summation-Guard, Solver-Guard, Thread-Enforcement.
- **T10** Diagnostik-Bänder ($T_{\text{sym}}, T_{\text{row}}$) als *Warn*-Signale (nicht als Fail), außer bei expliziten Profilen `strict_diagnostics=true`.
- **T11** Export-/Artefakt-Checks (Hash-Stabilität, Golden-Master-Hashes, Re-Run-Gleichheit).
- **T12** Kalibrier-Sweeps (Multiport/DtN) mit sauberer Trennung “Engineering-Brückentest” vs. fundamentale Claims.

T8: Sensitivitätsanalyse (deterministische Grids/Seeds). Dieser Test ist *keine* physikalische Behauptung, sondern ein robustes Engineering-Instrument: er quantifiziert die Stabilität von Outputs (Λ , d_n , Residuen, Fallback-Rate) gegenüber Policy-/Parameter-Variation bei strikt deterministischen Sweeps (keine Monte-Carlo-Sampling-Pfade ohne festes Seed).

T9: CI-Hard-Gates (Schema/Summation/Solver/Threads). In CI sind `OMP_NUM_THREADS=1` sowie Static-Checks (keine iterativen Solver; keine direkten Reduktionen) verpflichtend. Schema-Validation ist ein Fail-Gate.

T10: Diagnostik-Bänder als Warnungen (nicht Fail). Residuen $T_{\text{sym}}, T_{\text{row}}$ sind per Default *Warn*-Signale; sie werden nur in expliziten Profilen (`strict_diagnostics=true`) als Fail-Gate verwendet.

T11: Artefakt-/Golden-Master-Checks. Hashes (Inputs/Policy/Outputs) werden kanonisch berechnet und in Artefakten abgelegt; Re-Runs auf gleicher Toolchain müssen bit-identische Artefakte liefern. Cross-Platform gilt nur über Toleranzbänder.

4 Empirische Kalibrierung: Messbare Größen ohne Kontinuum-Input

Semantik (wichtig): Engineering-Validierung der Brücke $\neq$ Fundamentaltest. Die hier beschriebenen DtN/Multiport-Experimente validieren und kalibrieren primär die *Messbrücke* (Graph-Laplacian $\rightarrow$ Randresponse Λ , Kron/Schur/DtN, Widerstandsmetrik) und die *Optionen/Parameter der Spezifikation* innerhalb eines Widerstandsnetzwerk-Surrogats. Das ist **keine** direkte Evidenz für “Physik des Universums”, sondern eine kontrollierte Engineering-Validierung der emergenten Schnittstelle, auf die Pfeiler B–H später aufsetzen. Aussagen in Reports sind entsprechend zu formulieren. [1]

4.1 Warum DtN/Response als erste Kalibrierachse?

Die Dirichlet-to-Neumann-Map ist in EIT/Calderón-Problemen das zentrale Messobjekt: Randspannungen $\mapsto$ Randströme [4, 6, 7]. Im Diskreten ist dieselbe Struktur als Netzwerk-Response-Matrix zugänglich [3]. Damit ist DtN eine *direkte* Brücke zwischen (i) deinem Coarse-Graining (Schur/Kron) und (ii) realen Messdaten.

4.2 Messaufbau A: rein resistive Multiport-Netzwerke (DC)

Hardware. Aufbau eines physischen Resistor-Netzwerks, das einen endlichen Approximanten G_L (SG oder ST) realisiert, mit Randknoten B_0 als zugängliche Ports.

Messprotokoll.

1. Wähle L (z. B. 1–4) und realisiere Leitwerte $c(e)$ (IDEAL oder REAL-Update nach n Schritten).
2. Lege für jeden Port $i \in B_0$ definierte Spannungen V an (mehrere unabhängige Dirichlet-Randbedingungen).
3. Messe Ströme I an allen Ports; rekonstruiere die Response-Matrix Λ via linearem Fit $I = \Lambda V$.

Inferenzziel. Stufe 1 (Messbrücke): Fitte *realisierte Leitwerte* $\hat{c}(e)$ (und Mess-/Bauteiltoleranzen), so dass $\Lambda_{\text{model}}(\hat{c})$ die gemessene Λ_{meas} bestmöglich approximiert:

$$\theta^* = \arg \min_{\theta} \|\Lambda_{\text{model}}(\theta) - \Lambda_{\text{meas}}\|_W^2 + \text{Regularisierung}(\theta).$$

Die Identifizierbarkeit von Leitwerten aus Response ist in relevanten Klassen (z. B. kritische circular-planar Netzwerke) gegeben [3].

Identifizierbarkeit (kritisch): Stufe 1 $\neq$ Driver-Fit. Parameter wie $r_\star$, m_B oder σ_B wirken in der Definition primär über den Driver/Update-Schritt $c_n \mapsto c_{n+1}$, nicht über die statische Abbildung $c \mapsto \Lambda(c)$. Daher sind sie aus einer *einzelnen* statischen Messung ($c \equiv 1$, kein Update) typischerweise nicht eindeutig identifizierbar. Im Plan werden diese Parameter erst in **Stufe 2** gefittet: re-konfigurierbare Leitwerte (Hardware oder digitale Kopplung) erzeugen eine Folge $\{\Lambda_n\}_{n=0}^N$ bzw. $\{d_n\}_{n=0}^N$, und $\theta_B := (r_\star, m_B, \sigma_B, \dots)$ wird über diese Zeitreihe und mehrere Levels L joint geschätzt. [1]

4.3 Messaufbau B: RC-Netzwerke (Zeitantwort / Spektrum)

Durch Hinzufügen von Kapazitäten pro Knoten (oder pro Randport) erhält man ein lineares dynamisches System, dessen Relaxationszeitskalen eng mit dem Spektrum des Laplacians verknüpft sind (Diffusions-/Netzwerk-Analogie). Damit lassen sich *zeitliche Skalen* kalibrieren, die später in Pfeiler C (QDS/GKSL-Limit) benötigt werden [21, 19].

4.4 Messaufbau C: “digitale” Messungen (Simulation als Datenquelle)

Für frühe Iterationen ist eine *simulative* Datengenerierung (synthetische Messdaten mit Noise-Modellen) zulässig, um die Inferenzpipeline zu testen (nicht als “Bestätigung”). Diese Daten werden klar als synthetisch markiert; reale Hardware-Daten haben Priorität.

5 Welche freien Größen/Optionen sind sinnvoll kalibrierbar?

5.1 Parameterklassen (Priorität nach Pfeiler-Bau)

1. **Messbrücke / Hardware-Abbildung (Stufe 1):** Realisierte Leitwerte $\hat{c}(e)$, globale Skalierung, Kontakt-/Leitungswiderstände, sowie Messrauschen. *Kalibrierbar* aus statischer Multiport- Λ und (optional) Widerstands-/Energie-Checks über mehrere Randbedingungen und Wiederholungen. [3]
2. **Driver/Update-Parameter (Stufe 2):** z. B. $r_\star$ (Skalenweights α_k), m_B (Mismatch-Variante), σ_B (Orientierung) und ggf. weitere Driver-Amplituden. *Kalibrierbar* erst über *dynamische* Sequenzen $\{\Lambda_n\}$ bzw. $\{d_n\}$ unter realisiertem Update $c_n \mapsto c_{n+1}$ und über mehrere Levels L (Joint-Fit, Cross-Level-Consistency). [1, 8]
3. **OQS-Zeit-/Kopplungsskalen:** step-size h , Bad-Parameter, Lindblad-Raten bzw. repeated-interaction-Kopplungen. *Kalibrierbar* über Zeitreihen (RC oder OQS-Simulation/Experiment) und Stabilität/Stationarität [23, 22].
4. **Lokale DOF-Dimensionen (Pfeiler B):** minimale lokale Hilberträume/Algebren pro Zelle/Vertex. *Kalibrierbar* erst sekundär (über bessere Vorhersage von Response-Raten/Noise).

5.2 Nicht (oder nur sehr indirekt) kalibrierbar

- Rein numerische Stabilisierung ($\varepsilon_{\text{floor}}, \delta_R$) ist *fix* und nicht zu fitten.
- Visualisierungs-Einbettungen (2D/3D) sind nicht physikalisch.

6 Inferenz-Loop: “Calibrate $\rightarrow$ Narrow $\rightarrow$ Re-Validate”

6.1 Datenmodell und Qualitätskriterien

Jeder Messdatensatz enthält:

- Gerätekonfiguration (Messgerät, Auflösung, Temperatur, Verkabelung), Level L , REAL-Step n , Boundary-Set B_0 .

- Rohmessungen (V, I) , geschätzte Λ samt Unsicherheiten (Bootstrap).
- Fit-Ergebnis θ^* samt Konfidenz/Posterior und Gütemaßen (Residuals, Cross-Validation).

6.2 Entscheidungsregeln (“best choice”-Prinzip)

1. **Prefer minimal freedom:** nur Parameter freigeben, die durch Daten identifizierbar sind (sonst fixieren).
2. **Prefer pillar-support:** wähle Parametrisierungen, die *lokale Algebren* (Pfeiler B), *kontrollierte Limits* (Pfeiler C) und *thermodynamische Diagnostik* (Pfeiler D) unterstützen.
3. **Cross-level consistency:** ein Parameter-Set muss konsistent über mehrere L funktionieren (kein Overfitting an ein Level).

7 Roadmap Pfeiler B–H: Konkrete nächste Prüf-/Kalibrierschritte

7.1 Pfeiler B: Lokale Observablen/Algebren pro Zelle

- Definiere pro Zelle eine lokale Algebra $\mathcal{A}_{\text{cell}}$ und CP-Einbettungen/Reduktionen entlang der Hierarchie.
- Test: CP-Eigenschaften via Choi, Konsistenz unter Komposition (Stinespring/Choi) [15, 16].

7.2 Pfeiler C: Kontinuierliche Dynamik als Grenzregime (QDS/GKSL)

- Zeige numerisch (und soweit möglich analytisch), wann diskrete CPTP-Schritte eine Semigroup approximieren.
- Referenzrahmen: GKSL-Struktur [20, 19] und weak-coupling/Markov-Limits [21].
- Validierung: Generator-Fits aus diskreten Maps; Semigroup-Property-Tests.

7.3 Pfeiler D: Thermodynamik/Irreversibilität

- Implementiere Entropieproduktion (Spohn-Funktional) für QDS mit stationärem Zustand [22].
- Kalibrierung: stationäre Makrozustände und Relaxationsraten aus Zeitreihen; Robustheit/Metastabilität als Datenkriterium.

7.4 Pfeiler E: Causal-Cone-Ersatz (Lieb–Robinson-artig)

- Definiere Informationsausbreitung in der OQS-Schicht (Operator-Norm/Kommutator-Wachstum) und schätze effektive Geschwindigkeiten.
- Referenz: Lieb–Robinson Bound [26] und moderne Formulierungen [27].

7.5 Pfeiler F: Skalenfenster-Geometrie (Patch-Manifold-Nähe)

- Identifiziere Level-/Regionenfenster, in denen Widerstandsmetrik/Spektralgrößen stabil sind (keine globale Glättung behaupten).
- Bezug: Widerstandsformen und Laplace-Konstruktionen auf p.c.f. Sets [12, 14].

7.6 Pfeiler G: Detektor-Pfeiler (POVM/DRP-kompatibel)

- Operationalisiere Detektorelemente als Rand-/Zell-Observablen; “Response” als lineare Funktion von ρ (Kalibrierobjekt).
- Mathematischer Rahmen: CP-Instrumente/Kanäle [15, 18].

7.7 Pfeiler H: Symmetrie als späte Approximation

- Symmetrie wird nicht vorausgesetzt, sondern als emergentes Approximationsergebnis in Skalenfenstern getestet (Isotropie der Response, Approx.-Invarianzen).

7.8 Deterministische Default-Grids und Seeds (T12-Äquivalent)

Auch wenn die Spezifikation keine konkreten Parameterwerte vorgibt, muss die 1-Klick-Suite *deterministisch* starten können. Dazu definiert der Runner ein Default-Grid (nur für Sensitivität/Robustheit; nicht als Physik-Claim):

- **Levels:** $L_{\max} \in \{0, 1, 2, 3\}$ (Golden-Master für $L_{\max} \leq 2$).
- **Skalenfaktor:** SG: $r = 3/5$ (Fixwert), optionaler Sensitivitätssweep $r \in \{0.58, 0.60, 0.62\}$; ST: $r_{\star} \in \{0.5, 0.6, 0.7\}$ (bis belastbare externe Fixierung vorliegt).
- **Driver-B Optionen:** $m_B \in \{1, 2\}$ (falls zulässig), $\sigma_B \in \{-1, +1\}$, $\theta \in \{0.05, 0.1, 0.2\}$.
- **OQS-Stichproben (nur Profile `oqs_small/oqs_full`):**
Seeds $s \in \{0, 1, 2, 3, 4\}$ für `numpy.random.default_rng(s)`.

Alle Sweep-Punkte werden in einer kanonischen Reihenfolge abgearbeitet (lexikographisch nach Parameter-Key, dann Wert), und jeder Run schreibt ein eigenes `run_manifest.json` und `validation_report.json`.

8 Risiken, Gegenmaßnahmen und Pass/Fail

Risikoklasse 1: Nicht-Identifizierbarkeit. Gegenmaßnahme: Priorisiere DtN-basierte Messgrößen und kritische Netzwerkkonfigurationen, wo Leitwerte aus Response rekonstruierbar sind [3].

Risikoklasse 2: Numerik vs. Physik vermischt. Gegenmaßnahme: δ_R bleibt fix; jede physikalische Fit-Größe muss unabhängig davon stabil sein (Sensitivity-Reports).

Risikoklasse 3: OQS-Parameter zu frei. Gegenmaßnahme: starte mit minimalen repeated-interaction Kanälen und skaliere kontrolliert in GKSL-Limits [23, 19].

Pass/Fail-Minimum (für “Pfeiler A abgeschlossen”):

- 100% deterministische Reproduktion der Golden-Master-Artefakte.
- DtN/Kron-Schur-Identität erfüllt innerhalb vorgegebener Toleranzen.
- Widerstandsmetrik erfüllt Kernaxiome (Symmetrie, $R \geq 0$, $R(u, u) = 0$) und robuste Numerik-Residuals.
- CPTP-Tests bestehen (Choi/Trace) für definierte OQS-Schritte.

9 Anhang: Artefakt-Schemas (JSON Schema Draft 7)

Die folgenden Schemas sind **Pflicht** für CI/Runner. Der Runner validiert jede erzeugte Datei gegen diese Schemas. Zur deterministischen Hashbildung wird JSON *kanonisiert* (UTF-8, LF, Keys sortiert, keine Floating-NaN/Inf, keine Trailing-Spaces).

Definition von `policy_hash` (Pflicht, eindeutige Semantik). Zur Vermeidung von Mehrdeutigkeit wird im gesamten Runner festgelegt:

`policy_hash` $\equiv$ `numerical_policy_hash` := SHA256(canonical_yaml(numerical_policy.yaml)).

Damit verweisen `run_manifest.json` und jede `invalid_event`-Struktur auf exakt denselben Hash. (Optional kann ein zusätzlicher Diagnosewert `full_policy_hash` geloggt werden, z. B. Hash über `{config_hash, numerical_policy_hash}`; er ist aber *nicht* Bestandteil der Pflicht-Schemas.)

9.1 `run_manifest.json`

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "run_manifest",
  "type": "object",
  "required": [
    "spec_sha256",
    "plan_sha256",
    "config_hash",
    "numerical_policy_hash",
    "stack_fingerprint",
    "python_version",
    "profiles_run",
    "timestamp"
  ],
  "additionalProperties": false,
  "properties": {
    "spec_sha256": {
      "type": "string",
      "pattern": "^[a-fA-F0-9]{64}$"
    },
    "plan_sha256": {
      "type": "string",
      "pattern": "^[a-fA-F0-9]{64}$"
    },
    "config_hash": {
      "type": "string"
    },
    "numerical_policy_hash": {
      "type": "string",
      "pattern": "^[a-fA-F0-9]{64}$"
    },
    "numerical_policy_version": {
      "type": "string"
    },
    "stack_fingerprint": {
```

```

    "type": "object"
  },
  "python_version": {
    "type": "string"
  },
  "profiles_run": {
    "type": "array",
    "items": {
      "type": "string"
    }
  },
  "golden_masters": {
    "type": "array",
    "items": {
      "type": "string"
    }
  },
  "seeds": {
    "type": [
      "array",
      "null"
    ],
    "items": {
      "type": "integer"
    }
  },
  "timestamp": {
    "type": "string",
    "format": "date-time"
  }
}

```

9.2 validation_report.json

```

{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "validation_report",
  "type": "object",
  "required": [
    "spec_sha256",
    "run_id",
    "results",
    "invalid_flags",
    "invalid_events",
    "residuals",
    "diagnostics",
    "artifacts",
    "numerical_policy_hash",
    "timestamp"
  ],
  "additionalProperties": false,

```

```

"properties": {
  "spec_sha256": {
    "type": "string",
    "pattern": "^[a-fA-F0-9]{64}$"
  },
  "run_id": {
    "type": "string"
  },
  "numerical_policy_hash": {
    "type": "string",
    "pattern": "^[a-fA-F0-9]{64}$"
  },
  "timestamp": {
    "type": "string",
    "format": "date-time"
  },
  "results": {
    "type": "array",
    "items": {
      "type": "object",
      "required": [
        "test_id",
        "status",
        "details",
        "residues",
        "artifacts"
      ],
      "additionalProperties": false,
      "properties": {
        "test_id": {
          "type": "string"
        },
        "status": {
          "type": "string",
          "enum": [
            "PASS",
            "FAIL",
            "WARN",
            "SKIP"
          ]
        },
        "details": {
          "type": "string"
        },
        "residues": {
          "type": "object",
          "additionalProperties": {
            "type": "number"
          }
        },
        "artifacts": {

```

```

        "type": "array",
        "items": {
            "type": "string"
        }
    }
}
},
"invalid_flags": {
    "type": "array",
    "items": {
        "type": "string"
    }
},
"invalid_events": {
    "type": "array",
    "items": {
        "type": "object",
        "required": [
            "scope",
            "object_id",
            "reason",
            "test_id",
            "policy_hash",
            "stack_fingerprint",
            "when_utc"
        ],
        "additionalProperties": false,
        "properties": {
            "scope": {
                "type": "string"
            },
            "object_id": {
                "type": "string"
            },
            "reason": {
                "type": "string"
            },
            "test_id": {
                "type": "string"
            },
            "policy_hash": {
                "type": "string",
                "pattern": "^[a-zA-F0-9]{64}$"
            },
            "stack_fingerprint": {
                "type": "object"
            },
            "when_utc": {
                "type": "string",
                "format": "date-time"
            }
        }
    }
}

```

```

    }
  }
},
"residuals": {
  "type": "object",
  "additionalProperties": {
    "type": "number"
  }
},
"diagnostics": {
  "type": "object"
},
"artifacts": {
  "type": "array",
  "items": {
    "type": "string"
  }
},
"notes": {
  "type": "string"
}
}
}

```

10 Code-Verankerung: priorisierte Commits (Pflicht, um T0–T12 deterministisch zu machen)

Dieser Plan ist bewusst nicht nur *konzeptionell*, sondern benennt die minimalen Code-Artefakte, die im Repository vorliegen *müssen*, damit die Test-Suite (T0–T12) deterministisch und vollautomatisch (1-Klick) ausführbar ist.

1. Zentrale Summationsfunktion.

Implementiere `numeric/summation.py` mit `sum_kahan_lex(values, ids)` und erzwingen deren Nutzung in *allen* Aggregationen (z. B. Energien, Normierungen, Summen über Kanten/Zellen/Level). *Unit-Tests*: Permutationsinvarianz der Eingabereihenfolge $\Rightarrow$ identisches Ergebnis (bitwise innerhalb gleicher Toolchain).

2. Numerical Policy finalisieren und versionieren.

Committe `numerical_policy.yaml` mit bindenden Defaults: `Efloor_multiplier`, `AR_multiplier`, `cholesky_tol`, `Csym`, `Crow`, `TE_abs_multiplier`, `TE_rel_multiplier`, `h0`, `gamma`, `Nexpand`, `Nbisect`. Der Runner schreibt `numerical_policy_hash` als Pflichtfeld in `run_manifest.json`.

3. Deterministische LA-Wrapper (keine iterativen Solver).

Zusätzlich ist ein **Hard-Gate** erforderlich:

- **Static-Import-Check (Pre-commit/CI):**

`scripts/check_no_iterative_solvers.py` bricht ab, falls iterative Solver (z. B. `scipy.sparse.linalg.cg/gmres/bicgstab`) importiert oder verwendet werden.

Implementiere `linalg/wrappers.py` mit `det_cholesky(A) -> (success, L, reason)` und `det_solve_spd(A, b) -> (x, reason)`. Jede Exception wird deterministisch auf `invalid_reason`

gemappt (chol_fail:*, solve_fail:*). *Unit-Tests*: Nicht-SPD-Matrix $\Rightarrow$ chol_fail und valid=0.

4. Bisection/Bracketing in Code + Unit-Tests.

Implementiere

`real.lambda_bracket_bisect(F,policy)` mit deterministischem Expand/Bracket/Bisect gemäß Policy (h0,gamma,Nexpand,Nbisect,TE_abs,TE_rel) und den Reasons ok, ok_at_zero, no_bracket, budget_hit. *Unit-Tests*: je ein Testfall für jeden Reason; zusätzlich deterministische Wiederholbarkeit.

5. Logging und Schemas als CI-Hard-Gate.

Checke `schemas/run_manifest.schema.json` und `schemas/validation_report.schema.json` ein und erzwinge Schema-Validierung in CI. Minimaler CI-Check:

```
python -m jsonschema -i artifacts/run_manifest.json schemas/run_manifest.schema.json
python -m jsonschema -i artifacts/validation_report.json schemas/validation_report.schema.json
```

Schema-Verletzung $\Rightarrow$ CI-Fail (Audit-Trail-Integrität ist sonst nicht reproduzierbar).

References

- [1] antaris, *REAL-Graph_OQS-Definition* (vNext.3-canonical, 21.12.2025), interne Spezifikation (vom Nutzer bereitgestellt).
- [2] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed., SIAM, 2002.
- [3] E. B. Curtis, D. Ingerman, J. A. Morrow, Circular planar graphs and resistor networks, *Linear Algebra and its Applications* **283** (1998) 115–150.
- [4] A.-P. Calderón, On an inverse boundary value problem, in: *Seminar on Numerical Analysis and its Applications to Continuum Physics*, Sociedade Brasileira de Matemática, Rio de Janeiro, 1980.
- [5] L. Borcea, Electrical impedance tomography, *Inverse Problems* **18** (2002) R99–R136.
- [6] K. Astala, L. Päiväranta, Calderón’s inverse conductivity problem in the plane, *Annals of Mathematics* **163** (2006) 265–299.
- [7] G. Uhlmann, Electrical impedance tomography and Calderón’s problem, *Inverse Problems* **25** (2009) 123011.
- [8] F. Dörfler, F. Bullo, Kron reduction of graphs with applications to electrical networks, *IEEE Transactions on Circuits and Systems I: Regular Papers* **60**(1) (2013) 150–163.
- [9] P. Tetali, Random walks and the effective resistance of networks, *Journal of Theoretical Probability* **4** (1991) 101–109.
- [10] A. K. Chandra, P. Raghavan, W. L. Ruzzo, R. Smolensky, P. Tiwari, The electrical resistance of a graph captures its commute and cover times, *Computational Complexity* **6** (1996) 312–340.
- [11] D. J. Klein, M. Randić, Resistance distance, *Journal of Mathematical Chemistry* **12** (1993) 81–95.
- [12] J. Kigami, *Analysis on Fractals*, Cambridge University Press, 2001.

- [13] R. S. Strichartz, *Differential Equations on Fractals: A Tutorial*, Princeton University Press, 2006.
- [14] M. Fukushima, T. Shima, On a spectral analysis for the Sierpinski gasket, *Potential Analysis* **1** (1992) 1–35.
- [15] W. F. Stinespring, Positive functions on C^* -algebras, *Proceedings of the American Mathematical Society* **6** (1955) 211–216.
- [16] M.-D. Choi, Completely positive linear maps on complex matrices, *Linear Algebra and its Applications* **10** (1975) 285–290.
- [17] A. Jamiolkowski, Linear transformations which preserve trace and positive semidefiniteness of operators, *Reports on Mathematical Physics* **3** (1972) 275–278.
- [18] K. Kraus, *States, Effects, and Operations*, Lecture Notes in Physics 190, Springer, 1983.
- [19] G. Lindblad, On the generators of quantum dynamical semigroups, *Communications in Mathematical Physics* **48** (1976) 119–130.
- [20] V. Gorini, A. Kossakowski, E. C. G. Sudarshan, Completely positive dynamical semigroups of N -level systems, *Journal of Mathematical Physics* **17** (1976) 821–825.
- [21] E. B. Davies, Markovian master equations, *Communications in Mathematical Physics* **39** (1974) 91–110.
- [22] H. Spohn, Entropy production for quantum dynamical semigroups, *Journal of Mathematical Physics* **19** (1978) 1227–1230.
- [23] S. Attal, Y. Pautrat, From repeated to continuous quantum interactions, *Annales Henri Poincaré* **7** (2006) 59–104.
- [24] D. Petz, Quasi-entropies for finite quantum systems, *Reports on Mathematical Physics* **23** (1986) 57–65.
- [25] G. Lindblad, Completely positive maps and entropy inequalities, *Communications in Mathematical Physics* **40** (1975) 147–151.
- [26] E. H. Lieb, D. W. Robinson, The finite group velocity of quantum spin systems, *Communications in Mathematical Physics* **28** (1972) 251–257.
- [27] B. Nachtergaele, R. Sims, Lieb–Robinson bounds in quantum many-body physics, arXiv:1004.2086 (2010).